

Build & Deploy Runbook

Packetgeist — Cherwood Corporation, Division 02. Every remaining step, in order, from finishing the content to the site being live on your own hardware.

The plan in one picture

GitHub is the source of truth. Two things pull from it. Your DMZ container serves packetgeist.tarunc.com — that is the showcase, because you run the infrastructure. GitHub Pages serves a mirror that is always up even when the ThinkPad is off. One domain can only point at one of them, so the domain goes to your DMZ and Pages keeps its github.io URL.

Property	Domain	Served from	Repo
Packetgeist Division 02	packetgeist.tarunc.com	DMZ LXC + Cloudflare Tunnel	plainsboro-library-survey
Cherwood Health Division 01	cherwood.tarunc.com	already live	cherwood-health-lab
Personal	tarunc.com	already live	tarunc.com
Kindle digest	— (github.io)	GitHub Pages	kindle-daily-digest

Phase A — Finish the content (about 45 min)

Do this before anything is published. Everything below lands inside the repo folder.

A1 — Floor plans: already done

Both survey basemaps are in **assets/plans/**, extracted straight from the Maps/ folder inside your .netspu export — so they are the exact images the survey was plotted against. If you later want the children's level for the predictive model, save it there as `third-floor.png`.

A2 — Heatmaps → **assets/heatmaps/**

- ☐ Map menu → **Include/exclude networks** → deselect every non-library network first. Biggest visual-quality win available, costs two minutes.
- ☐ Display settings: **Min -85, Max -35** so the gradient spans your actual data instead of washing everything orange.
- ☐ Per floor, cycle Signal / SNR / SIR and use **Export to image**, not a screenshot. Six files.
- ☐ One extra: invert the filter and export a **neighbours-only** map. That is your external interference exhibit.
- ☐ SNR maps will be blank — no noise data was reported. Export and caption them as such rather than quietly omitting them.

A3 — Client evidence → **assets/screenshots/**

- ☐ Android Wi-Fi list showing 'Security: weak' — use as is.
- ☐ **Crop the Network Details shot to the two link-rate fields only.** The full screen exposes the library's client IP, gateway, DNS and subnet mask. Not yours to publish.

Phase B — GitHub (about 30 min)

B1 — Push the survey repo

Create an empty public repo named **plainsboro-library-survey**, then:

```
cd packetgeist-survey
git init
git add .
git commit -m "WiFi site survey: Plainsboro Public Library, Aug 2026"
git branch -M main
git remote add origin git@github.com:<you>/plainsboro-library-survey.git
git push -u origin main
```

- ☐ Description: 'Passive 802.11 site survey of a three-storey public library. 67 points, 1,523 observations, 8 APs. Coverage was never the problem.'
- ☐ Topics: wifi site-survey 802-11 wireless netspot rf cwna network-engineering
- ☐ Settings → Pages → Deploy from branch → **main / root**. Leave the custom domain field EMPTY — the domain is going to your DMZ, not here.
- ☐ Confirm the mirror loads at the github.io URL.

B2 — The other two on Pages

Same three clicks each. The goal is that all your work is browsable from one place.

Repo	Pages source	Note
cherwood-health-lab	main / root (or /docs)	Already has a live domain. Pages becomes a mirror that survives the ThinkPad being off — genuinely useful when a recruiter opens a link at 11pm.
kindle-daily-digest	main / docs	You already built Pages sites for this. Confirm the repo README matches the survey repo's shape: what it is, what it does, how to run it, what its limits are.

B3 — The index page

On **tarunc.com**, add a short section listing all three with one line each and a link. *That* page is the URL that goes on your resume — not three separate links.

Phase C — DMZ hosting (about 90 min)

Full detail, configs and commands are in **deploy/README.md** in the repo. This is the sequence and the reasoning.

C1 — The container

On Proxmox, create an **unprivileged Debian 12 LXC** on the DMZ VLAN. Name it pg-web. One core, 512 MB RAM, 4 GB disk — a static site needs nothing. Give it a static IP in the DMZ range.

Unprivileged matters: a container breakout from an unprivileged LXC lands you as an unprivileged user on the host, not root. For a box whose entire job is serving public traffic, that is the correct default and it is worth being able to explain why.

C2 — FortiGate policy

#	Source	Destination	Service	Action
1	pg-web (DMZ)	WAN	HTTPS 443	ALLOW — the tunnel's outbound connection
2	WAN	DMZ	any	DENY, logged — explicit, not implicit
3	pg-web (DMZ)	LAN	any	DENY — a web host has no business reaching your internal network

Rule 3 is the one that makes it a real DMZ rather than a VLAN you called a DMZ. Screenshot these policies — they belong in the Cherwood documentation.

C3 — nginx

```
apt update && apt install -y nginx git rsync curl
git clone https://github.com/<you>/plainsboro-library-survey.git /opt/packetgeist-src
mkdir -p /var/www/packetgeist
cp /opt/packetgeist-src/deploy/nginx-packetgeist.conf /etc/nginx/sites-available/packetgeist
ln -sf /etc/nginx/sites-available/packetgeist /etc/nginx/sites-enabled/
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx
```

The supplied config binds nginx to **127.0.0.1 only**. Nothing on the DMZ network can reach it directly — only the tunnel process on the same host can. It also sets security headers and a content security policy scoped to the one external thing the page loads (Google Fonts).

C4 — Cloudflare Tunnel

```
curl -L
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb -o
/tmp/cf.deb
dpkg -i /tmp/cf.deb
cloudflared tunnel login
cloudflared tunnel create packetgeist
cloudflared tunnel route dns packetgeist packetgeist.tarunc.com
# edit /etc/cloudflared/config.yml from the example in deploy/
cloudflared service install
systemctl enable --now cloudflared
```

The DNS record creates itself. tunnel route dns writes the CNAME into Cloudflare for you. Do not also add a manual A or CNAME record for packetgeist — you will end up with two conflicting answers.

If you already run a tunnel for cherwood.tarunc.com, you have a choice: a second independent tunnel (simpler to reason about, one failure domain per site) or one tunnel with two ingress hostnames (fewer moving parts). The

example config shows the second form commented out. For a lab you are documenting, separate tunnels are easier to explain.

C5 — Auto-deploy

```
cp /opt/packetgeist-src/deploy/packetgeist-deploy.* /etc/systemd/system/  
systemctl daemon-reload  
systemctl enable --now packetgeist-deploy.timer  
systemctl list-timers packetgeist-deploy.timer
```

Every ten minutes it fetches, compares HEAD to origin, and does nothing if they match. On a change it resets, rsyncs into the webroot excluding .git and deploy/, and reloads nginx. Push to GitHub, wait ten minutes, the DMZ site updates itself.

You already run a Flask receiver in the lab. Pointing a GitHub webhook at it for instant deploys instead of polling is a reasonable later upgrade — but the timer has no inbound attack surface at all, which suits a DMZ host better. Start here.

C6 — Verify

```
curl -sI http://127.0.0.1/ | head -1  
systemctl status cloudflared --no-pager  
curl -sI https://packetgeist.tarunc.com | head -1  
# from OUTSIDE your network:  
nmap -Pn -p 80,443 <your-wan-ip>
```

The nmap should find nothing. That negative result is the entire design, and it is a screenshot worth keeping.

Phase D — The predictive model (one evening)

Method is in **planning/README.md**. The condensed version:

- NetSpot Planning, both floor plans from assets/plans/, scale calibrated identically to the survey.
- Draw walls from the materials in section 2.3 of the report — brick cavity over CMU for the wings, low-E curtain wall for the central volume, gypsum partitions, dense masonry at the entry.
- Place the eight APs at surveyed positions with their current channels from data/ap-inventory.csv.
- **Calibrate.** Compare predicted against measured in data/scan-points.csv. If it predicts -60 where you measured -50, tune wall attenuation until they agree, and write down what you changed. This step is the difference between a model and a guess.
- Duplicate, apply the recommended plan, render both, drop the images in planning/.

What the comparison shows. Not better coverage — coverage is already 100%. It should show *fewer co-channel radios audible per location* while holding the -67 dBm floor. Saying that accurately is more impressive than claiming a coverage win that does not exist.

Phase E — Publish and send (about an hour)

E1 — Process narrative

One page on the site or LinkedIn. What you expected (dead zones), what you found (none, and the opposite problem), how you got there (the export is a database, not a picture), what you got wrong on the way (you trusted 'map coverage 75%, good' from six points before working out it measures spread, not density), and what you'd do differently.

E2 — Send it to the library

Two staff gave you permission. Sending the report closes the loop and turns a practice exercise into work with a recipient. Attach the findings PDF only — not the data pack, not the raw export. Link the site if they want more. Draft email is in the Completion Guide.

Final check

- ☐ No internal IP addresses anywhere published — including in nginx logs you screenshot.
- ☐ No identifiable patrons in any image.
- ☐ No third-party architectural photography republished.
- ☐ Architectural Record credited for plans and product schedule.
- ☐ F-04 still labelled unverified everywhere it appears.
- ☐ The two exclusions still read as scope decisions, not apologies.
- ☐ Every number on the site traceable to a CSV in data/.
- ☐ GitHub Pages custom-domain field left empty on the survey repo.
- ☐ nmap from outside returns nothing.